

iOS Swift Take-Home Assignment

Build a small iOS application that demonstrates modern iOS development practices, clean architecture, and Swift proficiency.

The focus of this task is not pixel-perfect UI, but rather code quality, architecture, scalability, and maintainability.

Authentication

- Username/password login
- Biometric authentication using Face ID / Touch ID via LocalAuthentication
- Persistent login session using Keychain and/or secure app storage
- Input validation, loading states, and error states
- Lightweight animation using Lottie for iOS or native SwiftUI animation

Products

- Fetch products from <https://dummyjson.com/products>
- Display paginated product list
- Product details screen
- Search, sorting, and filtering
- Offline caching support

Favorites

- Add/remove favorite products
- Persist favorites locally
- Undo remove action

Settings

- Dark/light mode support
- Language switching (English/Hebrew)
- Logout functionality

CRUD

- Add/Edit/Delete products locally
- Reset local changes from API

AI-Assisted Development Requirement

Candidates may use AI development tools such as Claude Code, Codex, Cursor, GitHub Copilot, or similar tools during the assignment. However, the candidate remains fully responsible for the final implementation and code quality.

- Include an AI Usage Report in the README
- Describe which AI tools were used
- Explain what AI assisted with
- Describe what was implemented or reviewed manually
- Include at least 3 meaningful prompts used during development
- Explain how correctness and code quality were verified

Expectations

- Clean and maintainable architecture

- Good state management
- Proper error handling
- Offline-first thinking
- Reusable and testable code
- Smooth and responsive UI
- Clear separation between UI, domain, data, and platform-specific layers

Required Tech Stack

Category	Requirement
Language	Swift
UI	SwiftUI and UIKit - demonstrate controls/screens using both frameworks
Architecture	MVVM, MVI, or The Composable Architecture (TCA)
Async	Swift Concurrency: async/await, Task, AsyncSequence where relevant
Dependency Injection	Constructor injection, lightweight DI container, Factory, or Resolver
Networking	URLSession with Codable, or Alamofire if justified
Local Storage	SwiftData or Core Data; UserDefaults/AppStorage for simple settings; Keychain for secure session data
Pagination	Manual cursor/limit pagination or a reusable paging abstraction
Images	AsyncImage, URLSession-backed image loading, or Kingfisher/Nuke if justified
Testing	XCTest for unit tests; optional XCUITest for UI flows

Deliverables

- Source code
- README with setup instructions
- Architecture explanation
- Testing credentials
- AI Usage Report
- Notes about trade-offs, assumptions, and known limitations

Estimated completion time: 8-12 hours.

Prioritize architecture, stability, testability, and code quality over visual polish.